

Ahmad Masood (04072213041)
The Main Motivation and Project Idea

Motivation

In today's digital marketplace, choosing the right product has become surprisingly *difficult*. Users often switch between *multiple* websites to compare prices and still remain uncertain about quality. This process is time-consuming, confusing, and inefficient, especially when information is scattered across different platforms.

The idea behind the Price and Quality Comparison Application (PQC) emerges from this very gap. The goal is simple but meaningful: to reduce effort, remove confusion, and give users clarity. By combining intelligent search (through NLP) with automated data collection, the system aims to create a space where users can compare products effortlessly, seeing both price and quality side by side and make decisions with confidence.

Project Overview

The Price and Quality Comparison Application (PQC) is designed as a web-based system that brings together product information from multiple sources into one unified platform. Instead of forcing users to search across different websites.

A user begins by entering a product name whether in English or Urdu. The system interprets this input using Natural Language Processing. From there, a web scraping engine collects relevant product data from various online platforms and available digital physical stores.

This data is organized. Prices, ratings, availability, and other details are structured in a way that makes comparison user friendly rather than confusing. Users can quickly scan through options, explore details, and identify what suits them best.

Characteristics / Features

1. Smart Multilingual Search

Users are not restricted by language. Whether the query is in *Urdu or English*, the system adapts. It processes, translates, and refines the input so that users get accurate and relevant results without needing to adjust their natural way of searching.

2. Multi-Source Data Collection

Instead of depending on a single platform, the system gathers information from multiple sources. This ensures that users are not seeing a limited view but a broader, more realistic picture of the market.

3. Real-Time Price Comparison

Prices are pulled and presented in real time, allowing users to immediately identify the most cost-effective options. No manual comparison is needed; the system handles it seamlessly.

4. Quality and Rating Analysis

Price alone rarely tells the full story. By including *ratings* and *reviews*, the system helps users understand the quality behind each option, encouraging smarter decisions rather than cheaper ones.

5. Filtering and Sorting

Users remain in control. They can narrow down results based on their preferences whether its price, rating, or category and sort them in a way that aligns with their priorities.

6. User-Friendly Interface

The interface is kept simple, clean, and responsive. The aim is not to impress with complexity but to provide clarity, and users feel comfortable navigating the system.

System Design Overview

The PQC application follows a three-tier architecture, separating concerns to maintain clarity and scalability.

The **presentation layer** acts as the point of interaction. This is where users search, explore, and compare products. It focuses entirely on delivering a smooth and responsive experience.

The **application layer** handles the core logic. It processes user input, communicates with the NLP module, triggers the scraping engine, and organizes the results.

The **data layer** manages storage. It keeps track of product data, stored in db, cached results, and user interactions, ensuring that information is both accessible and consistent.

System Implementation Overview

The implementation of the PQC system brings together modern technologies in a way that balances performance with flexibility.

On the frontend, **Next.js** is used to build a responsive and dynamic interface.

The backend is powered by **Node.js**, which handles server-side operations and manages communication between different components.

For data collection, Puppeteer is used as a web scraping tool.

MongoDB is chosen as the database due to its flexibility in handling diverse product data. It stores not only product details but also cached results to improve performance.

The **NLP** module plays a critical role by interpreting user queries, handling language translation, and extracting meaningful keywords.